

Arquitectura de ordenadores 2024/2025

Práctica 3

Jorge Jiménez Oropesa, Antonio Moroño Moreno

Grupo 1322, pareja 15

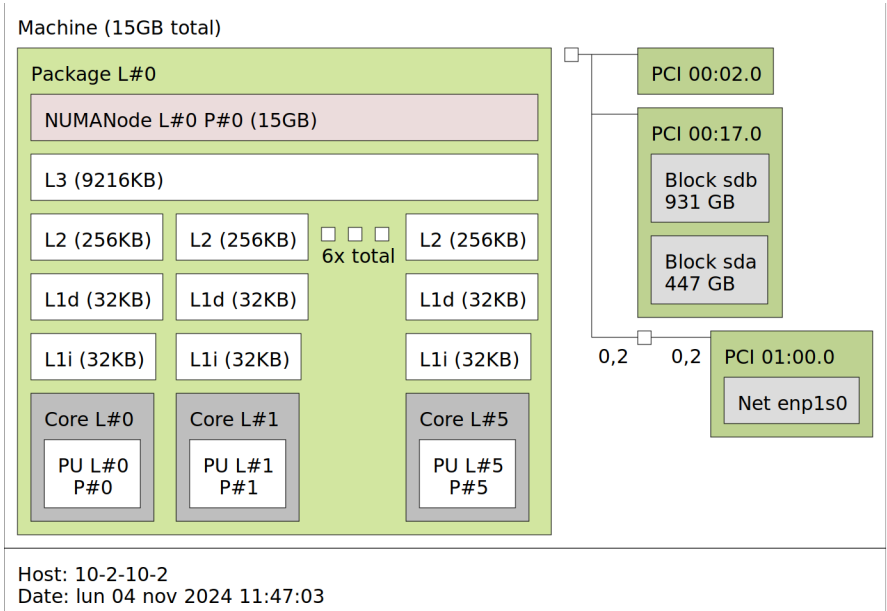
Ejercicio 1

Apartado 1

Empleando el comando `getconf -a | grep -i cache` obtenemos la siguiente información sobre la caché:

LEVEL1_ICACHE_SIZE	32768
LEVEL1_ICACHE_ASSOC	
LEVEL1_ICACHE_LINESIZE	64
LEVEL1_DCACHE_SIZE	32768
LEVEL1_DCACHE_ASSOC	8
LEVEL1_DCACHE_LINESIZE	64
LEVEL2_CACHE_SIZE	262144
LEVEL2_CACHE_ASSOC	4
LEVEL2_CACHE_LINESIZE	64
LEVEL3_CACHE_SIZE	9437184
LEVEL3_CACHE_ASSOC	12
LEVEL3_CACHE_LINESIZE	64
LEVEL4_CACHE_SIZE	0
LEVEL4_CACHE_ASSOC	
LEVEL4_CACHE_LINESIZE	

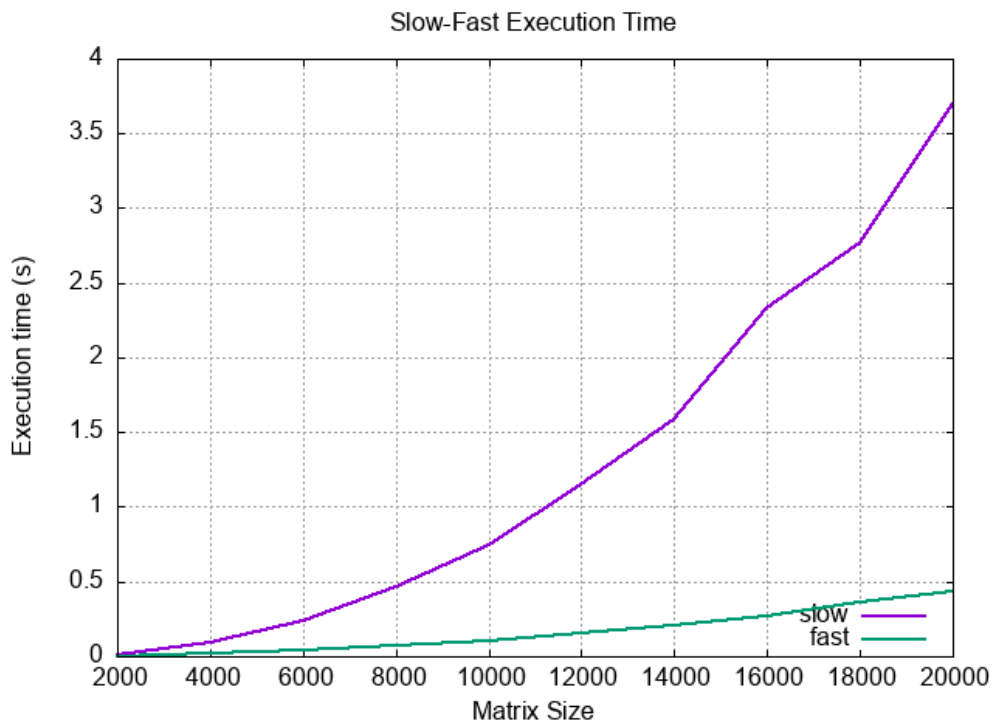
Empleando la instrucción `lstopo`, por otro lado, se nos presenta esta información de forma más visual.



Podemos ver que hay 6 cores, cada uno con una memoria caché L1i de 32 KB, otra caché L1d de 32 KB, y otra caché L2 de 256 KB. Además, hay una caché común a todos los cores de tipo L3 con 9216 KB. Además, podemos ver que la separación de datos e instrucciones está presente únicamente en el nivel L1, ya que se separan las cachés entre L1i (instructions) y L1d (data).

Ejercicio 2

A continuación se muestra una gráfica el tiempo promedio que cada versión del programa ha tardado en realizar los cálculos:



Es importante realizar varias veces la toma de medidas porque al encontrarnos en un entorno experimental, los tiempos pueden verse afectados por factores externos, como otros procesos que se estén ejecutando simultáneamente en el sistema operativo o cambios en la frecuencia del reloj. Realizando varias pruebas reducimos el impacto de estos factores.

a. Por qué para matrices pequeñas los tiempos de ejecución de ambas versiones son similares, pero se separan según aumenta el tamaño de la matriz?

Esto se debe a que para la suma de los elementos de una matriz, si consideramos la operación de acceso a memoria para obtener el valor de un elemento, esta tiene una complejidad $O(N^2)$. Esto se debe a que para una matriz cuadrada con N elementos por fila/columna, la matriz tiene N^2 elementos, y para la suma se accede una vez a cada elemento. Por lo tanto, si hacemos que el acceso a memoria sea M veces más rápido, entonces la suma de los elementos, cuyo tiempo de ejecución viene principalmente determinado por la velocidad de acceso a memoria, será aproximadamente M^2 veces más rápido. Así que si la gráfica de slow es de la forma x^2 , entonces la gráfica de fast será aproximadamente $\frac{x^2}{M^2}$, luego la diferencia entre estas dos es $x^2 \frac{M^2-1}{M^2}$, y por lo tanto la diferencia es cuadrática y consecuentemente creciente.

b. ¿Cómo se guarda la matriz en memoria, por filas (los elementos de una fila están consecutivos) o bien por columnas (los elementos de una columna son consecutivos)?

Las matrices se almacenan en memoria como un array en el que las filas se encuentran ordenadas de forma adyacente, de modo que los elementos de una fila son consecutivos.

c. ¿Cuál es el tamaño máximo de matriz que cabe en la caché L1 del ordenador que está utilizando? Recuerde que se guardan “Double” (punto flotante de doble precisión)

Tenemos 6 cores cada uno de los cuales dispone de una cache L1d de 32 KB. Un double ocupa 8 bytes, así que simplemente tenemos que dividir L1d entre 8 para obtener el número de doubles que caben en la caché L1 conjunta.

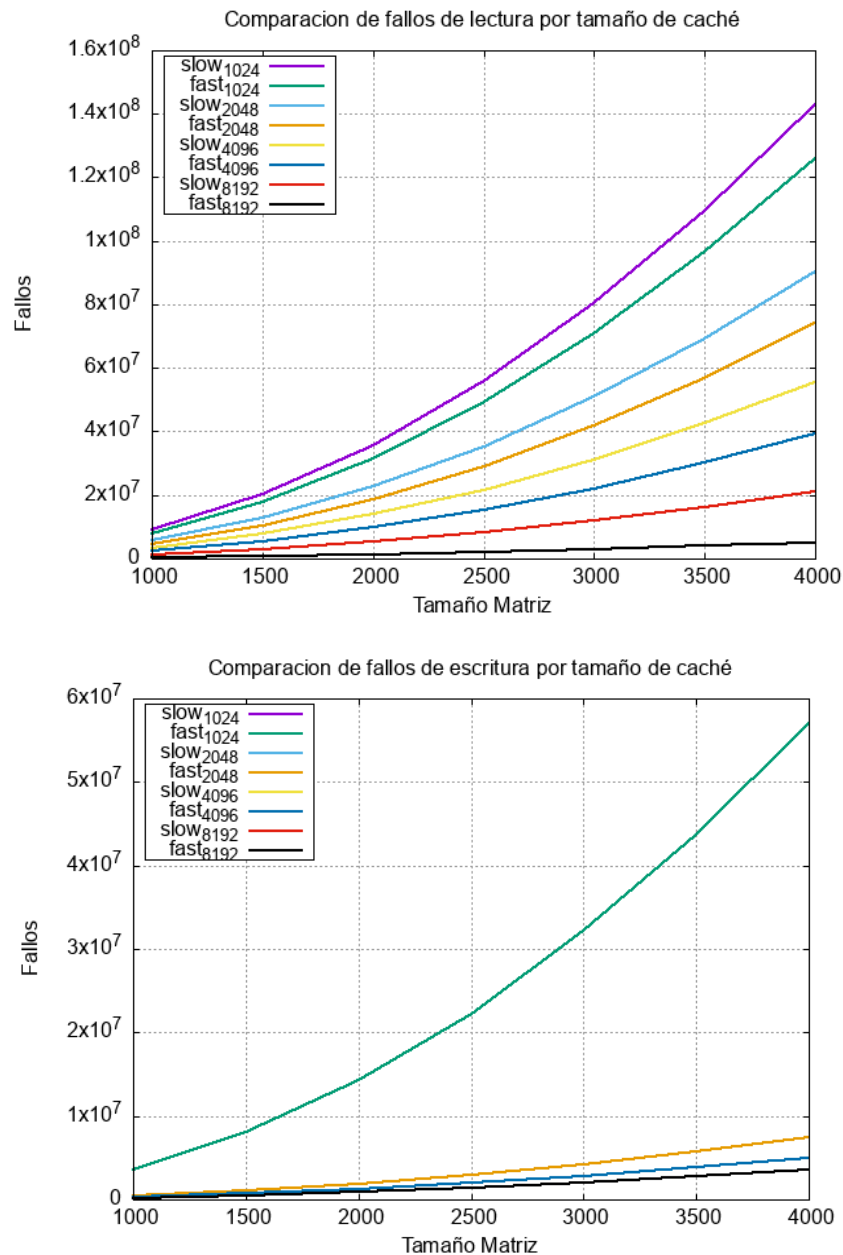
$$\frac{32 \cdot 2^{10} \cdot 6}{8} = 24.576 \text{ doubles}$$

Con este número de doubles la máxima matriz que podemos almacenar en la caché sería de dimensión 156x156:

$$\sqrt{24.576} = 64\sqrt{6} \approx 156$$

Ejercicio 3

Estas son las gráficas que hemos obtenido:



a. ¿Se observan cambios en la gráfica al variar los tamaños de caché para el programa slow? ¿Y para el programa fast?

Sí. Observamos que al aumentar el tamaño de caché, disminuye el número de fallos, ya que se pueden almacenar más datos, luego es menos probable que haya un conflicto. Esto es cierto tanto para lectura como escritura.

b. ¿Varía la gráfica cuando se fija en un tamaño de caché concreto y compara el programa slow y fast? ¿A qué se debe cada uno de los efectos observados?

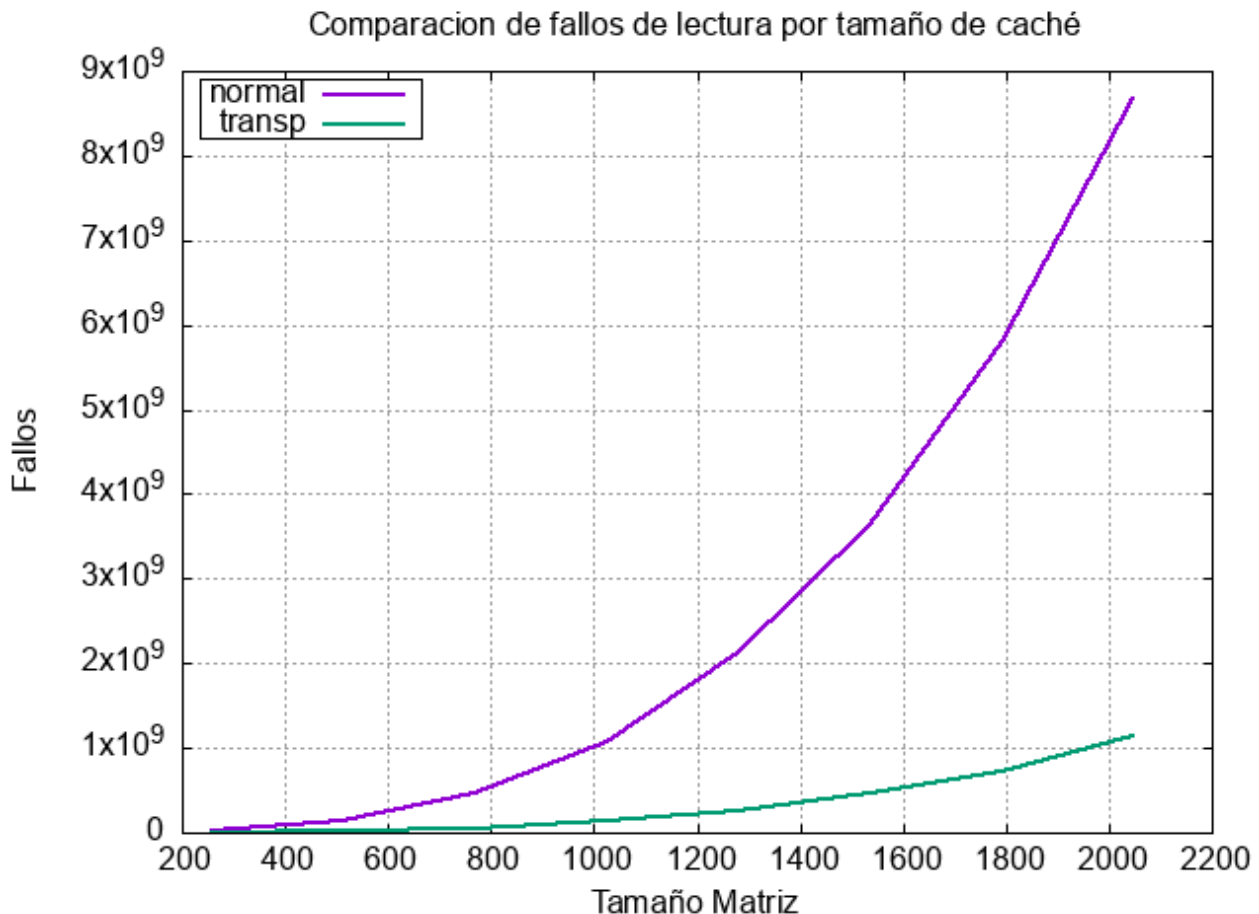
En cuanto a la diferencia entre slow y fast, podemos ver que en la comparación de lectura, dado un cierto tamaño de caché, la versión fast comete menos fallos de caché, gracias a que sigue el

row-major order. Los fallos de escritura, en cambio, no se ven afectados por la versión (fast o slow), ya que ambos escriben en una única variable, así que el orden de acceso a los valores de la matriz no afecta a la escritura, ya que no se escribe sobre la matriz.

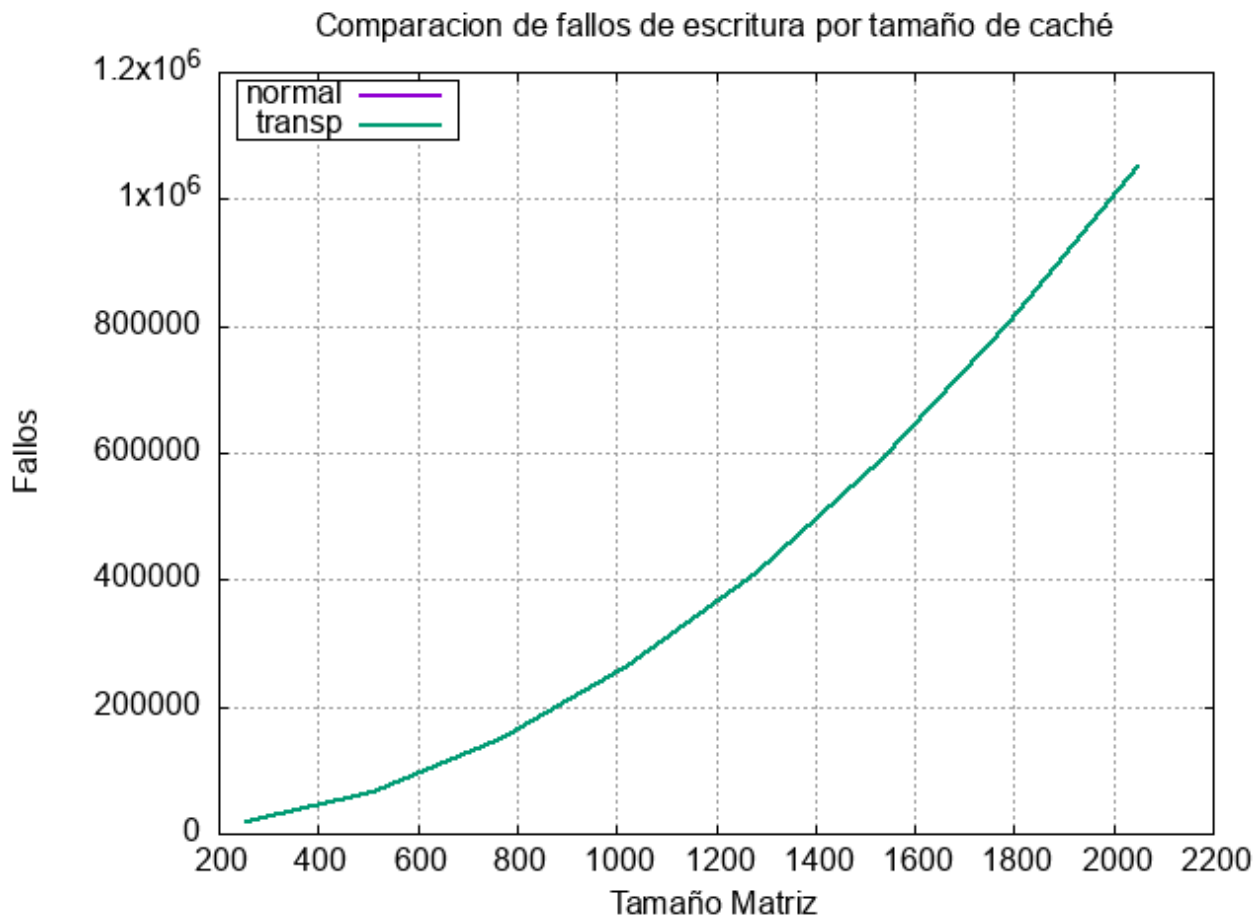
Ejercicio 4

Hemos creado un script que unifica los apartados 1 y 2, de modo que mide los tiempos de ejecución que resultan de ejecutar varias veces cada versión del programa con distintos tamaños de matriz, y calcula la media. Además, como también implementa el apartado 2, ejecuta cada versión del programa con cachegrind, ahora sí una única vez. Los datos los guarda en *mult.dat*. Con esto, graficamos los resultados obtenidos mediante GNUplot, y estas son las tres gráficas que obtenemos, y que ahora procederemos a comentar:

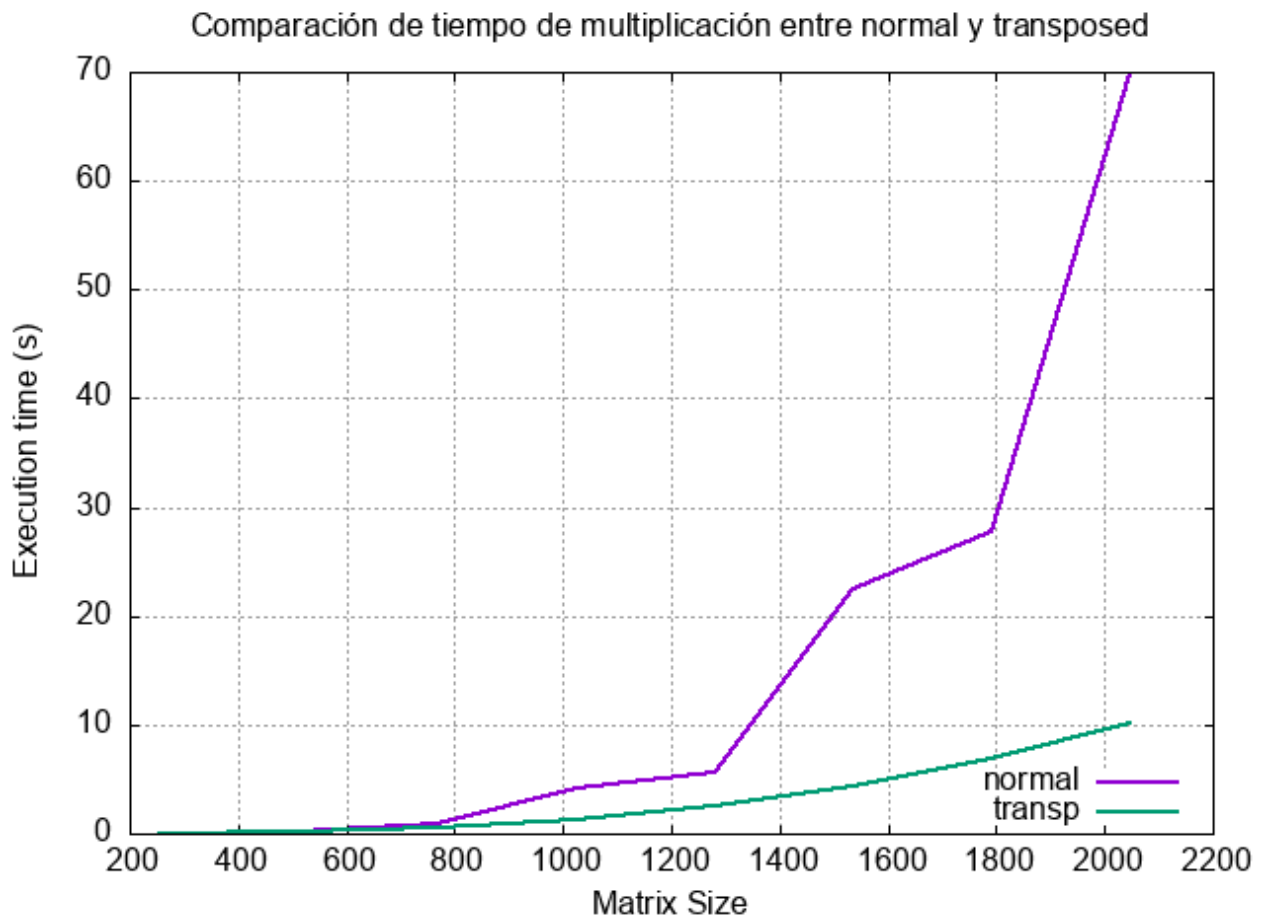
Nota: hay un pequeño error en el título. Realmente, sería comparación por tamaño de matriz.



Como era de esperar, observamos que la versión normal del programa comete más fallos de lectura. La versión slow recorre la segunda matriz del producto por columnas, lo que genera muchos fallos de caché, como ya vimos en el apartado previo. La versión fast, en cambio, la recorre por filas gracias a la transposición, por lo que reduce el número de fallos aprovechando la naturaleza row-major order de las cachés. Por supuesto, a mayor tamaño de matriz se realizan más lecturas, por lo que aumenta el número de fallos. Y como la complejidad del producto de matrices es cúbica en cuanto a lecturas, las gráficas siguen ese comportamiento.



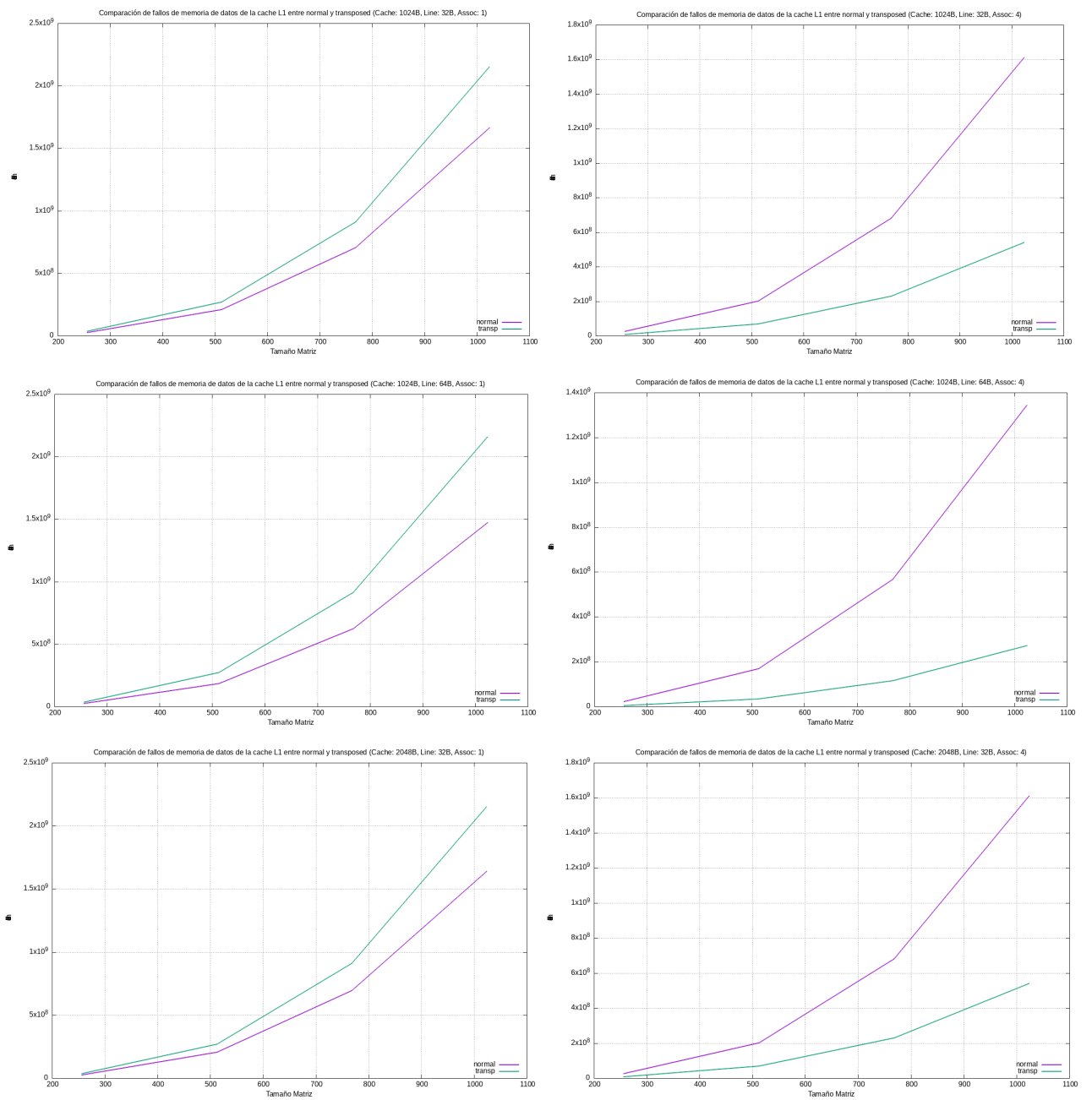
En cuanto a los fallos de escritura, podemos observar que se cometen exactamente el mismo número de fallos. Esto se debe a que el orden en el que se escriben los resultados del producto matricial en la matriz destino es el mismo para slow y fast, por lo que es de esperar que la escritura se comporte de la misma forma. Por supuesto, a mayor tamaño de matriz se realizan más escrituras y por lo tanto ocurren más fallos. Y como el número de escrituras es el número de elementos en la matriz destino, es decir, $N \times N$, la gráfica tiene un comportamiento cuadrático.

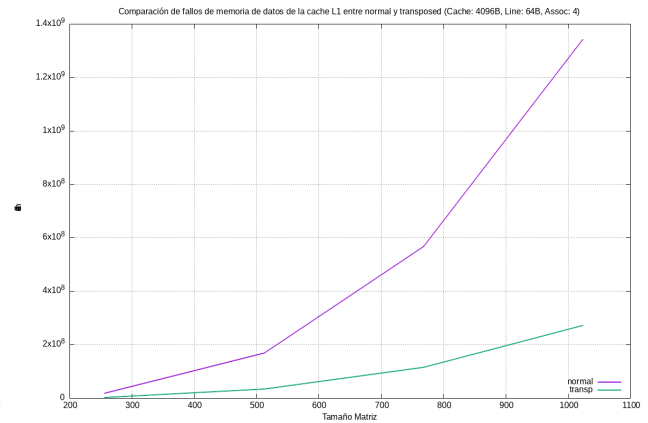
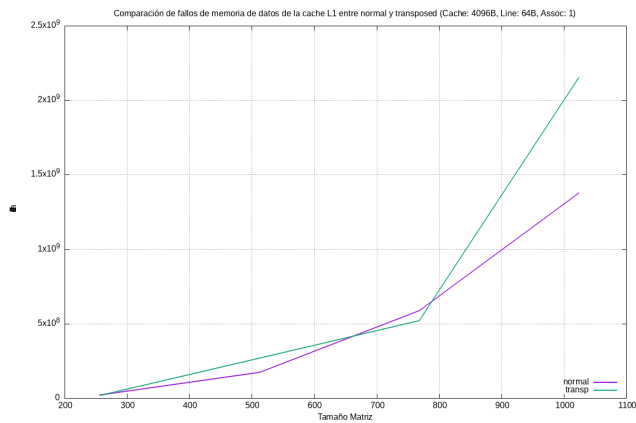
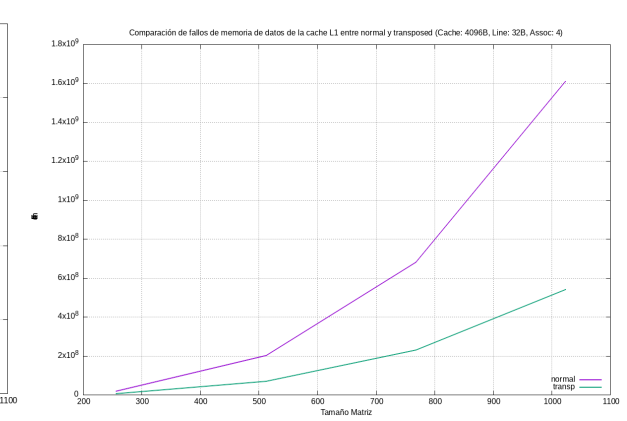
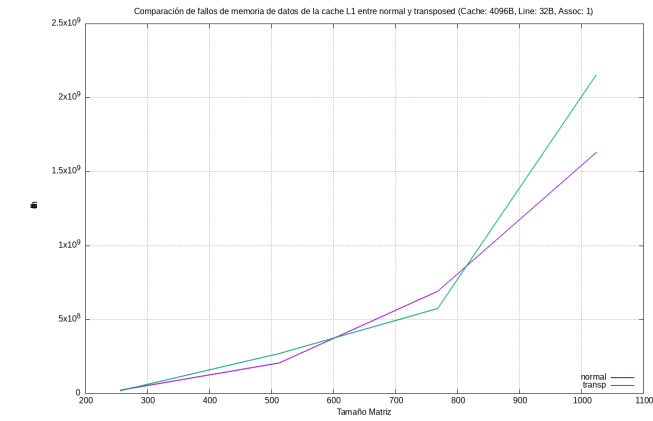
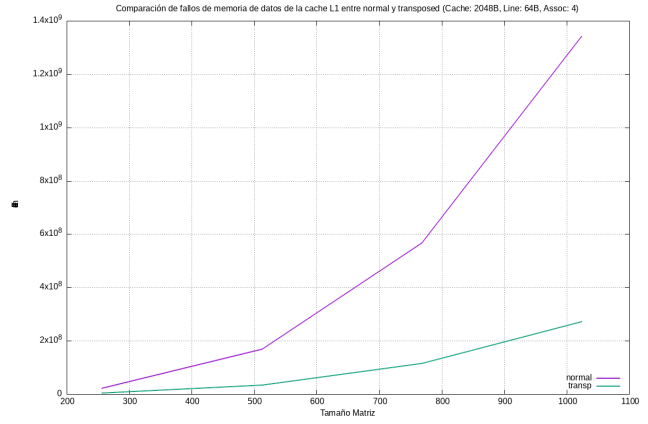
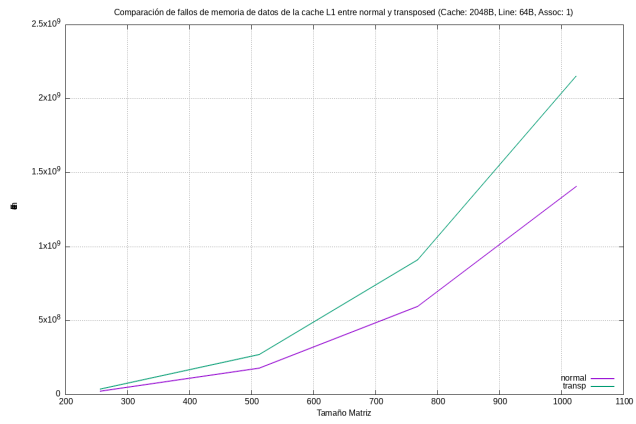


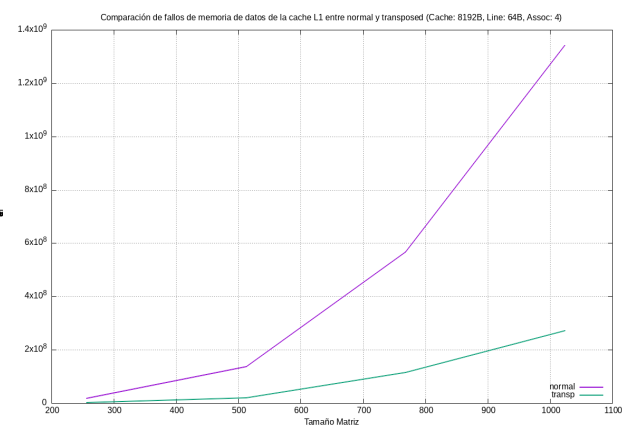
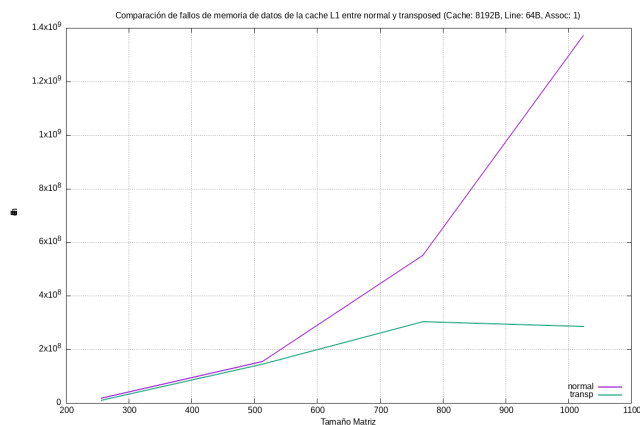
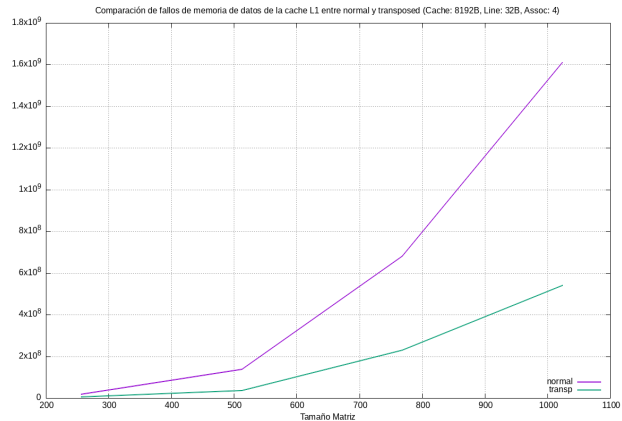
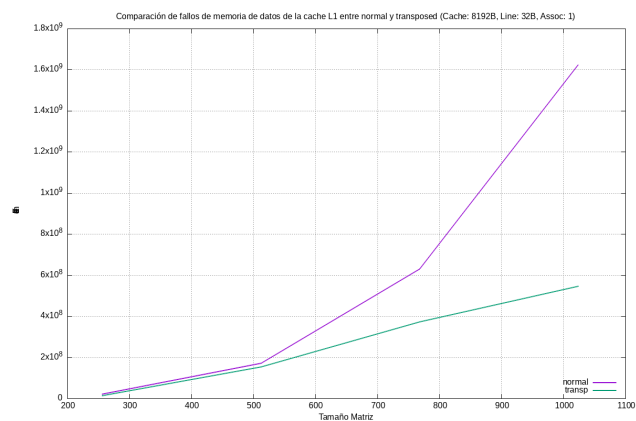
En cuanto al tiempo de ejecución, como era de esperar, la versión normal resulta más lenta que la transpuesta, ya que aunque el número de fallos de escrituras es el mismo, el la versión normal comete muchos más fallos de lectura, y esto lastra tremendamente el rendimiento. Por otro lado, las gráficas siguen una tendencia aproximadamente cúbica, que es coherente con el cálculo de matrices, que es de complejidad cúbica respecto a lecturas, sumas y multiplicaciones escalares.

Ejercicio 5

Hemos simulado la ejecución para tamaños de caché de 1KB, 2KB, 4KB y 8KB, para tamaños de línea de 32B y 64B, y con asociatividad de 1, 4 y 8. A continuación se muestran las gráficas con los resultados obtenidos:







Comentarios:

- Para empezar, hemos decidido no incluir las gráficas con asociatividad 8, ya que en todos los casos resultaban indistinguibles a las versiones con asociatividad 4. Si atendemos a los .dat, los datos son distintos, pero varían tan poco que al graficarlo no se distingue.
- Con asociatividad 1, cada posición de memoria solo puede mapearse a un único bloque en la caché. Esto genera un mayor número de fallos, especialmente cuando los tamaños de línea y de caché son pequeños. En estas condiciones, la versión fast puede llegar a ser menos eficiente que slow. Esto ocurre porque la principal ventaja de fast es cargar filas completas en la caché y acceder a ellas de manera contigua, aprovechando el row-major order. Sin embargo, si el tamaño de la caché es muy reducido y, además, la asociatividad es única, esta optimización se ve limitada. Nuestra hipótesis es que no todos los elementos de una fila caben en la caché, lo que obliga a realizar frecuentes recargas. Por otro lado, fast lleva a cabo la transposición de la matriz, que también genera fallos en la caché. Al aumentar la asociatividad a 4, el número de fallos disminuye considerablemente, como reflejan las gráficas. Esto beneficia especialmente a la versión fast, cuya ventaja reside en una gestión eficiente de la caché para aprovechar su optimización por filas. Por otro lado, la ejecución con asociatividad 8 ofrece una mejora marginal respecto a la asociatividad 4 y, como ya mencionamos, estas diferencias no son apreciables en las gráficas.
- Tamaño de caché: como ya pudimos observar en el apartado 3, aumentar el tamaño de caché disminuye el número de errores, especialmente para la versión fast. Esto se debe a que, al contar con una memoria más amplia, caben más datos y es menos probable que ocurra un error. En el caso de fast, que accede a las matrices por filas aprovechando el

row-major order, una caché más grande permite que más filas completas o partes significativas de estas permanezcan en la memoria, minimizando los fallos al recorrerlas.

- Tamaño de línea: el uso de tamaños de línea mayores beneficia principalmente a la versión fast, ya que le permite realizar menos cargas, y aprovechar al máximo las que hace gracias al acceso por filas, que son contiguas en memoria. Slow, en cambio, no puede aprovechar esto, por lo que realizará aproximadamente las mismas cargas.

Como conclusión, podemos extraer que para poder exprimir las optimizaciones de fast, necesitamos disponer de unos requisitos de caché mínimos, pues de lo contrario no se saca partido a la localidad espacial de la memoria gracias al row-major order. Una vez disponemos de un hardware algo más capaz, fast resulta significativamente más rápido que slow.

Así, hemos encontrado en este ejemplo una bonita demostración de que para obtener el máximo rendimiento posible, centrarse únicamente en el software descuidando el hardware (ejecutar fast con cachés muy limitadas) o lo contrario (utilizar una caché potente pero con slow) dará lugar a unos resultados alejados de lo óptimo. La verdadera virtud, probablemente, se halle en el equilibrio.